Modeling the Cooling of Air in a Can

Assignment 2

SCIE 001 Physics

Noah Virjee
45515863

November 21, 2025

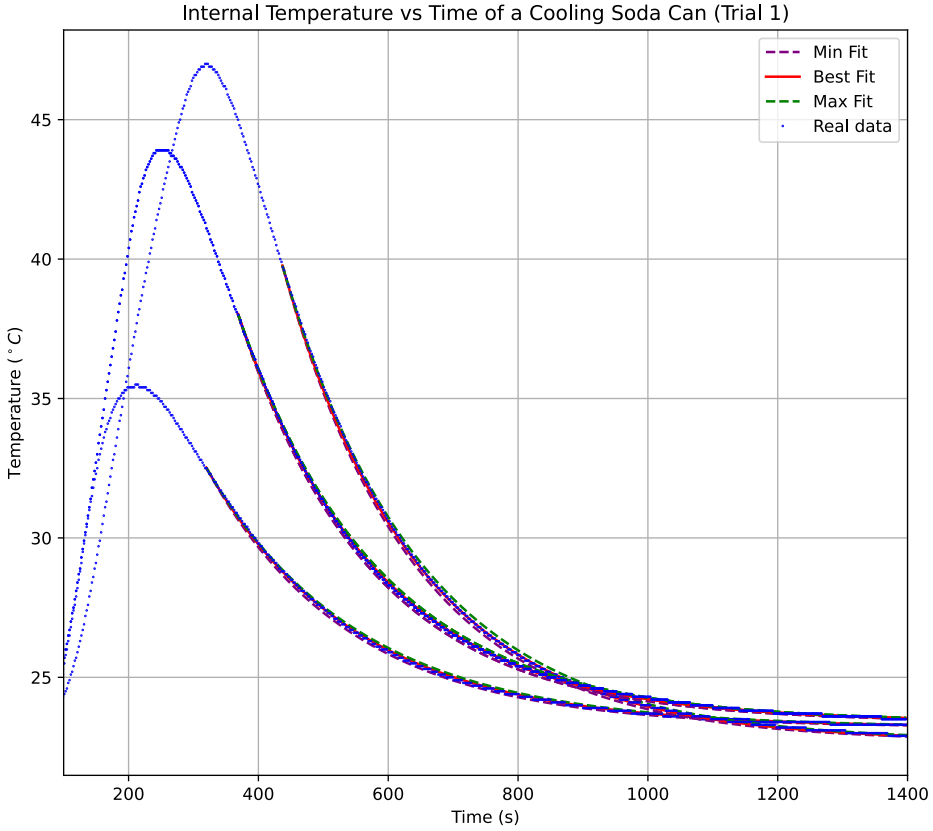

Figure 1: Graph of the internal temperature over time for Trial 4.

Contents

Appendix

1. Introduction

I used a space heater to heat a standard 355 ml aluminum soda can, then let it cool down while monitoring internal temperature with an arduino and DHT11 temperature sensor.

I then fit a Euler method-based model to this data to determine the emissivity constant and convention coefficient of the can.

The can weighed $(0.01246 \pm 2.887 \times 10^{-6})$ kg, with a radius of $(0.0331 \pm 1.00 \times 10^{-3})$m , and an effective height of $(0.112 \pm 3.25 \times 10^{-3})$m.

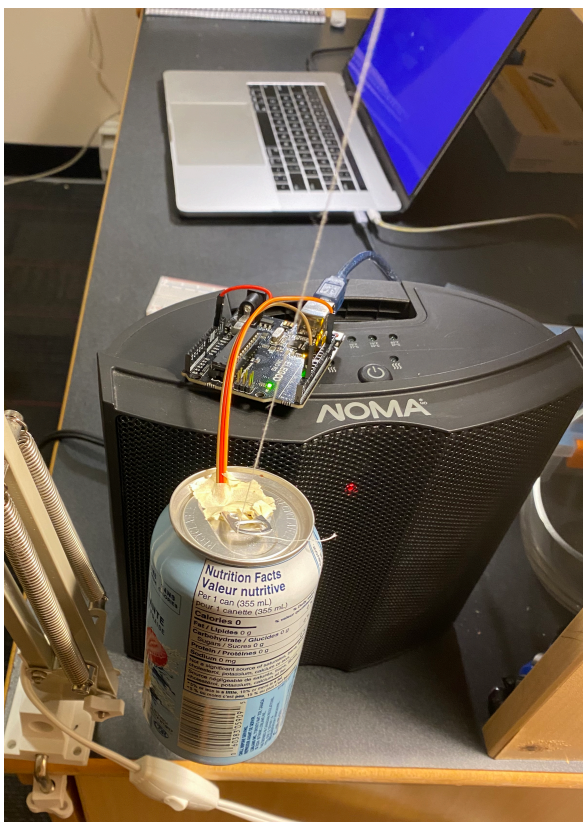

Figure 2: Image of the experimental setup. The can was hung from a string to limit heat flow to mostly radiative and convective and not conductive. Tape was used as a cover to limit air movement outside the can.

Figure 3: An image of the tissue box that was dropped onto the sensor on a scale with 0.01g precision.

2. Results

Data was fit to a computational model starting at the inflection point where the slope of the cooling graph began to decline. Parameters k_c and ε were manually adjusted using an interactive fitter (see Appendix A) to reduce χ^2 score (see Appendix B) between experimental data and model while keeping a good fit.

The constants used are shown in Table 2, and the final parameters and fit data is shown in Table 1.

Uncertainties in k_c and ε were calculated using gaussian error:

$$\delta k_c = \frac{k_c\text{max} - k_c\text{min}}{4}$$
$$\delta\varepsilon = \frac{\varepsilon\text{max} - \varepsilon\text{min}}{4}$$

Final convection coefficient and emissivity came out to $k_c = 0.369 \pm 0.011 \text{W}/m^2/\text{K}$ and $\varepsilon = 0.22537 \pm 0.0025$ by averaging the best k_c's and propagating the uncertainties.

$$k_c = \frac{k_{c1} + k_{c2} + k_{c3}}{3} \qquad \varepsilon = \frac{\varepsilon_1 + \varepsilon_2 + \varepsilon_3}{3}$$
$$\delta(k_c) = \left(\frac{1}{3}\right)\sqrt{(\delta k_{c1})^2 + (\delta k_{c2})^2 + (\delta k_{c3})^2} \qquad \delta(\varepsilon) = \left(\frac{1}{3}\right)\sqrt{(\delta\varepsilon_1)^2 + (\delta\varepsilon_2)^2 + (\delta\varepsilon_3)^2}$$

There are two differential equations we used to model heat flow.

$$\frac{dQ_c}{dt} = -k_c A(T - T_{\text{amb}}) \qquad \text{for convective heat flow}$$

Where:
- A is area in m^3
- k_c is the convection coefficient in $Wm^{-2}K^{-1}$
- T is the current temperature of the system in K
- T_{amb} is the ambient temperature of the environment in K

$$\frac{dQ_r}{dt} = -A\varepsilon\sigma\left(T^4 - T_{\text{amb}}^4\right) \qquad \text{for radiative heat flow}$$

Where:
- A, T, and T_{amb}, are the same as before.
- ε is the emissivity of the surface of the object:

The equation for the change in total heat was:

$$\therefore \frac{dQ_{\text{total}}}{dt} = \frac{dQ_c}{dt} + \frac{dQ_r}{dt}$$

To determine the change in temperature, we rearrange the formula for heat capacity.

$$C_p = \frac{\Delta Q}{\Delta T}$$
$$\Delta T = \frac{\Delta Q}{C_p}$$

Where C_p is the heat capacity of our system at constant pressure in JK^{-1} (instead of the usual $J\text{mol}^{-1}K^{-1}$).

Rewriting with differentials:

$$\frac{dT}{dt} = \left(\frac{1}{C_p}\right)\frac{dQ_{\text{total}}}{dt}$$
$$\therefore \frac{dT}{dt} = \left(\frac{1}{C_p}\right)\left(\frac{dQ_c}{dt} + \frac{dQ_r}{dt}\right)$$

For Euler-based modeling in python, this looks like:

```
dHc = kc * A * (Tn-Tamb) * dt
dHr = eps * sig * A * (Tn**4 - Tamb**4) * dt
Tn = Tn - (dHc + dHr)/heatcap
```

This model assumes that there was no conductive heat loss, heat capacity of the system stays constant, no hot air left the container, and ambient temperature is static.

2.1. Parameters and Fit Data

Trial Number	Estimated k_c W/m^2/K	Relative Uncertainty in k_c W/m^2/K	Estimated ε	Relative Uncertainty in ε	χ^2	ΔT across wall of can K
1	0.394 ± 0.037	0.094	0.2248 ± 0.0051	0.023	0.56	0.011
3	0.4130 ± 0.0026	0.0064	0.2059 ± 0.0091	0.044	0.73	0.0063
4	0.301 ± 0.069	0.23	0.2454 ± 0.0099	0.040	1.7	0.013

Table 1: The trial number, estimated k_c and ε and their relative uncertainties, χ^2 of fit, and temperature difference across wall of can.

2.2. Constants used in code:

Name	Value	Uncertainty	Link
π	3.141592653589793		
Radius of Can	0.0331 m	$\pm 1.00 \times 10^{-3}$	
Height of Can	0.112 m	$\pm 3.25 \times 10^{-3}$	
Volume of Can	$0.000355 \mathrm{m}^{-3}$		Determined from the ml rating on the can.
rhoair	1.225 kg m^{-3}		link
rhoAl	2712 kg m^3		link
mAl	0.01246 kg	$\pm 2.887 \times 10^{-6}$ kg	
thickness of Al	0.095 m		
conductivity of Al	237 W/m K		link
Heat capacity of air	1005 J/kg/K		link
Heat capacity of aluminum	897 J/kg/K		link
0 degrees Celsius in Kelvin	273.15 K		
Stefan-Boltzmann Constant	5.67e-8 W/m^2/K^4		

Table 2: The constants used in the code.

2.3. Graphs

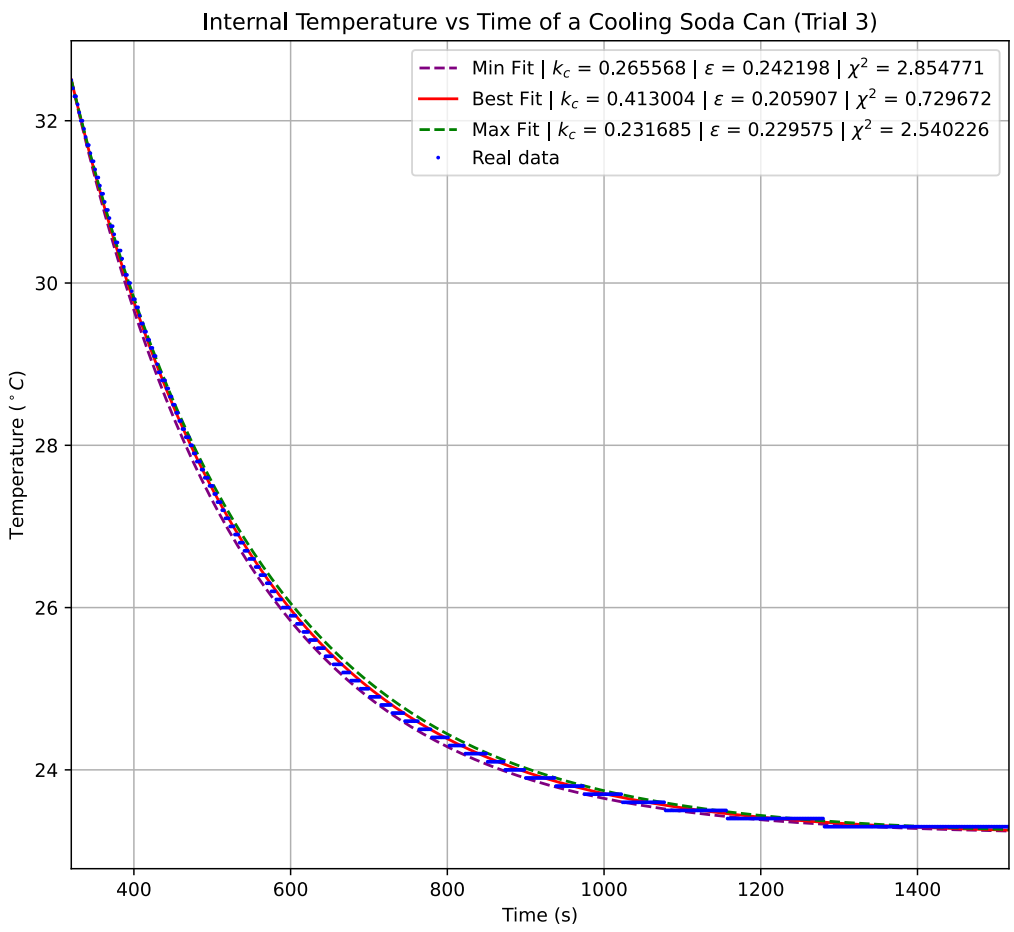

Figure 4: Graph of the internal temperature of the can over time for Trial 3.

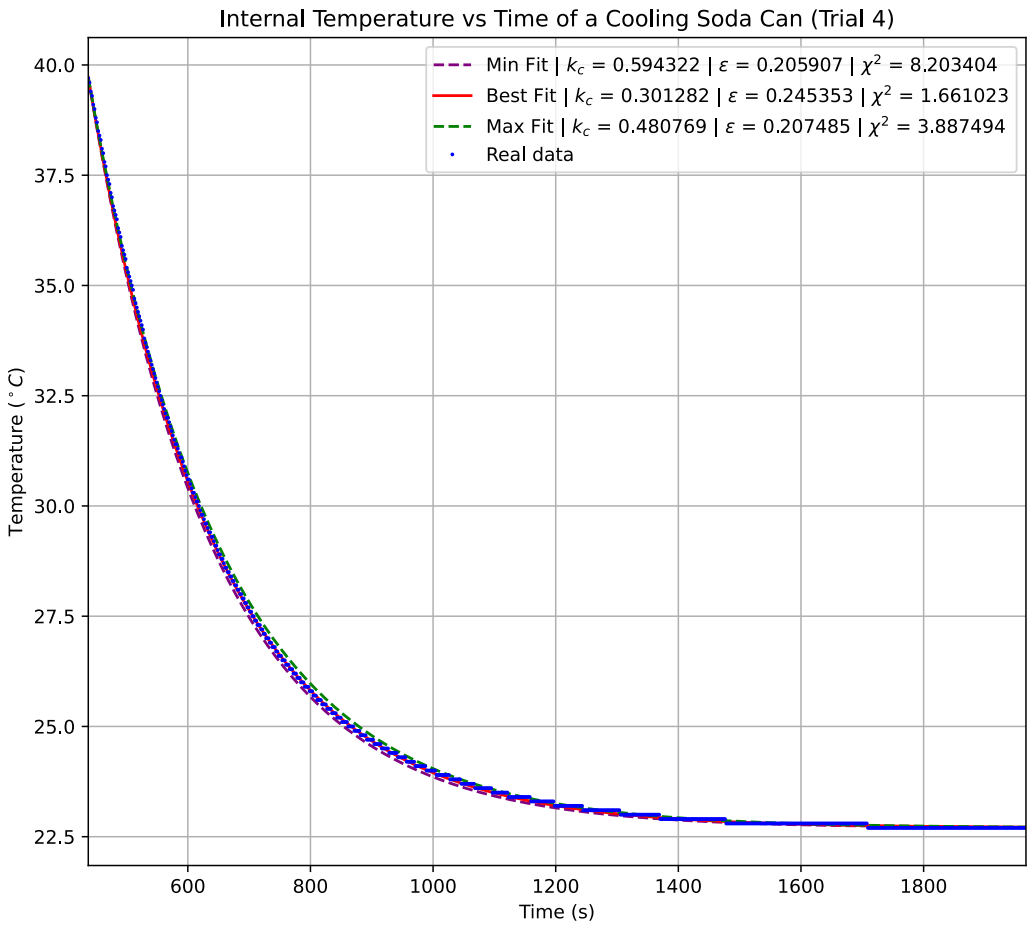

Figure 5: Graph of the internal temperature of the can over time for Trial 4.

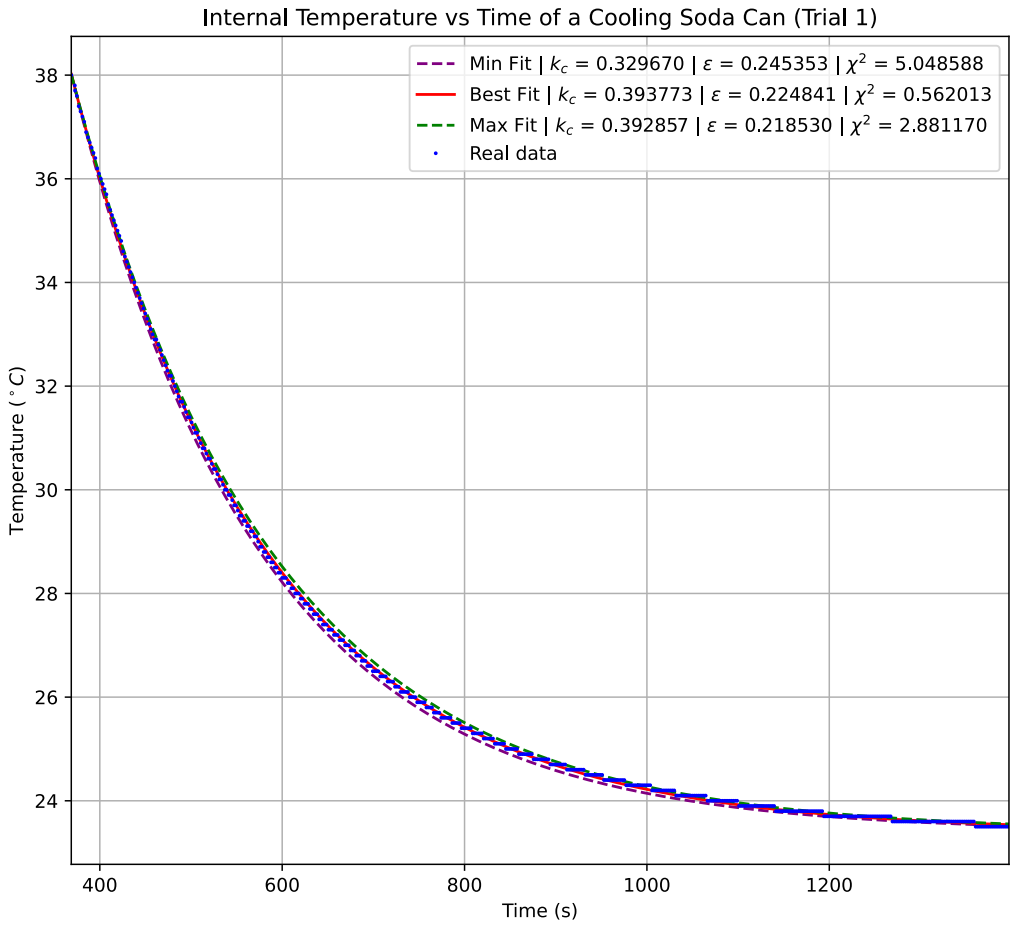

Figure 6: Graph of the internal temperature of the can over time for Trial 1.

3. Discussion

Our graphs show the internal temperature of the can over time plotted against an Euler model to predict the temperature of the can given its temperature at one point.

Our expanded differential equation is:

$$\frac{dT}{dt} = \left(\frac{1}{C_p}\right)\left(-k_c A(T - T_{\text{amb}}) + \left(-A\varepsilon\sigma\left(T^4 - T_{\text{amb}}^4\right)\right)\right)$$

$$\frac{dT}{dt} = \left(\frac{1}{C_p}\right)\left(-k_c AT + k_c AT_{\text{amb}} - A\varepsilon\sigma T^4 + A\varepsilon\sigma T_{\text{amb}}^4\right)$$

Therefore if $T > T_{\text{amb}}$, $\frac{dT}{dt} < 0$ and so the temperature decays over time till $T = T_{\text{amb}}$, $\frac{dT}{dt} = 0$ and temperature has equalized. This is exactly what we see in our numerical approximation.

The ε does not agree well across the trials, but the k_c agrees well, this is evident by their t-prime scores being greater than 1 about 1 respectively in Table 3.

Trial Pair	t-prime Score of their k_c	t-prime Score of their ε
3 and 1	1.061	3.145
3 and 4	1.515	3.810
1 and 4	2.837	3.988

Table 3: The calculated t-prime scores for the best estimated k_c and ε of each pair of trials.

Based on the here, and that my can was painted, my emissivity is reasonable. Aluminum Heavily Oxidized was listed with $\varepsilon = 0.2 - 0.31$ and mine was $\varepsilon = 0.2254 \pm 0.0025$.

Relative to here showing k_c of aluminum $= 6.87$ W/m^2/K, my convection coefficient of $k_c = 0.369 \pm 0.011$W/m^2/K is is unreasonable (off by a factor of 614).

We graph the change temperature across the wall of our can over time:

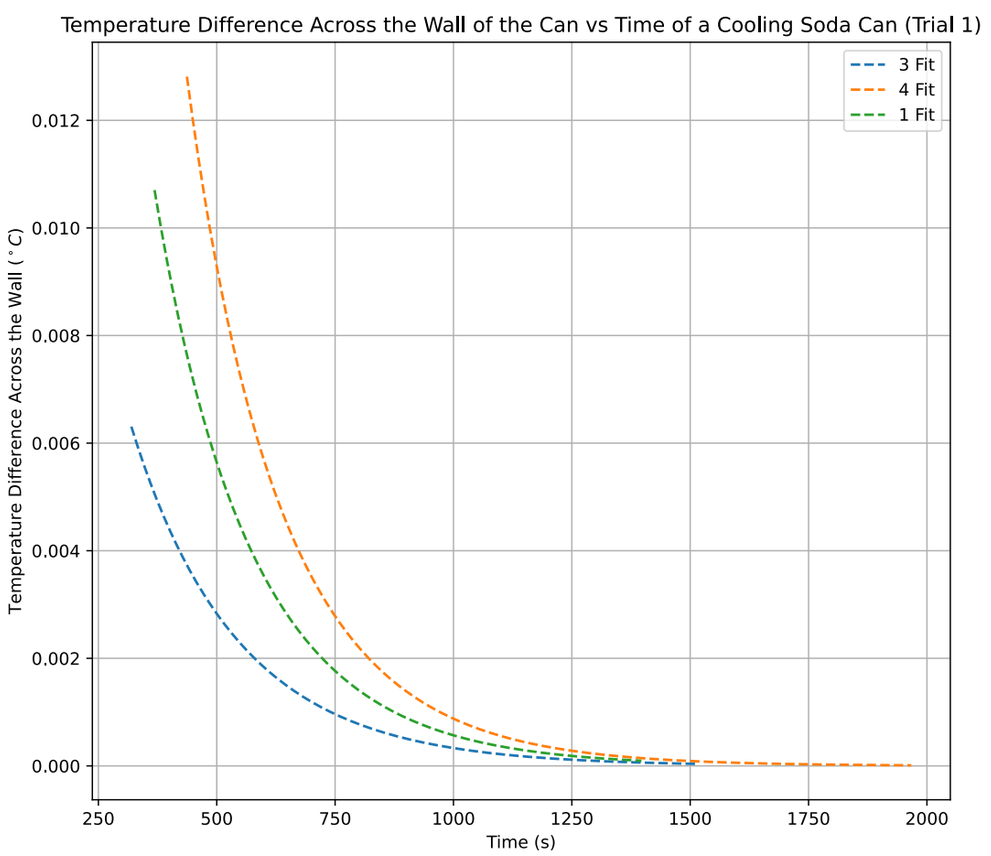

Figure 7: A graph of the simulated change in temperature across the wall over time.

Figure 7 shows aluminum is a good conductor, as heat flows out of the can readily until it equalizes with ambient temperature, then heat stops flowing.

This is also a result of the second law of thermodynamics, more specifically that if no work is done heat flows across a temperature gradient.

A Interactive Fitter

Note: I included this appendix just for fun because I found it interesting. You do not have to read it for marking purposes.

I wrote some custom code to do interactive fitting, you can find it here: https://github.com/blucardin/SCIE001Physics/blob/assignment_2/interactive_fitter.ipynb

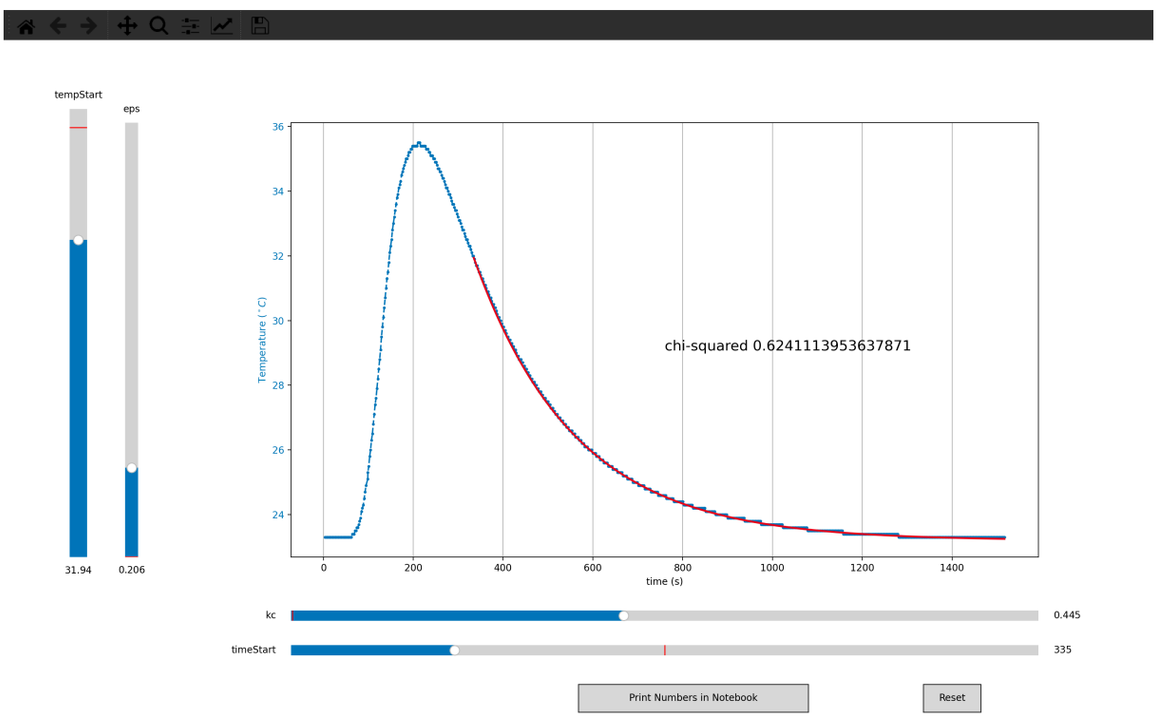

Figure 8: An image of the interactive fitting application.

B Calculating χ^2

For the fitter to work, I needed to calculate χ^2, but since the datapoints were not aligned in time I couldn't compute the metric normally.

So I wrote some code that did some interpolation between the points, you can find it below.

```
def chiSquared(x1s, y1s, x2s, y2s):
    # since they are not aligned on the time axis, we have to do some interpolation.
    # (the time axis for the experimental temperatures varies, so even if we matched
them, they wouldn't fit)

    sum = 0
    for x1, y1 in zip(x1s, y1s):
        observed_value = y1

        point_after_index = np.searchsorted(x2s, x1)
        point_before_index = point_after_index - 1


        if point_after_index >= len(y2s):
            # we are at the end of the list, so just use the last interpolation
            point_after_index -= 1
            point_before_index -= 1

        m = (y2s[point_after_index] - y2s[point_before_index])/
(x2s[point_after_index] - x2s[point_before_index])

        interpolated_value = (m * (x1 - x2s[point_before_index])) +
y2s[point_before_index]

        sum += ((observed_value - interpolated_value)**2)/ (interpolated_value)

    return sum
```